

React

En omfattande och strukturerad guide över arkitektur, koncept och kodmönster

Kort historik: React skapades ursprungligen av Jordan Walke, en mjukvaruingenjör på Facebook (numera Meta), och släpptes med öppen källkod 2013. Målet var att lösa prestandaproblem i stora applikationer med mycket dynamisk data genom att introducera en virtuell DOM och en deklarativ komponentbaserad arkitektur. Idag är React världens mest använda bibliotek för webbutveckling.

1. Kärnkoncept: Komponenter i djupet

Hela React bygger på idén om **komponenter**. En komponent är i grund och botten en återanvändbar, isolerad bit av gränssnittet. I modern React är en komponent helt enkelt en vanlig JavaScript-funktion som returnerar kod som ser ut som HTML (vilket kallas JSX).

När du skapar en komponent definierar du dess utseende och logik på ett ställe. Därefter kan du använda den komponenten precis som en egen HTML-tagga var som helst i din applikation.

Komponentinstanser (Samma namn, olika data)

Ett av de absolut viktigaste koncepten att förstå är att varje gång du använder en komponent, skapar React en helt ny **instans** av den komponenten. De må heta samma sak i koden, men de fungerar som helt oberoende kopior.

```
// Här använder vi SAMMA komponent tre gånger.  
// Trots att alla är en 'UserCard', är de helt oberoende av varandra.  
export default function App() {  
  return (  
    <div>  
      <UserCard name="Anna" age={25} />  
      <UserCard name="Kalle" age={32} />  
      <UserCard name="Maria" age={41} />  
    </div>  
  );  
}
```

Även om "Kalle" skulle klicka på en knapp inuti sitt `<UserCard />` som ändrar en färg (sitt eget state), förblir Anna och Marias kort helt opåverkade.

Kodexempel: Hur UserCard kan se ut

Här nedanför ser du hur själva kortet är uppbyggt. Den tar emot `name` och `age` som Props utifrån, men håller reda på sina egna knapptryck via sitt isolerade State.

```
import { useState } from 'react';

// Komponenten tar emot 'name' och 'age'
export default function UserCard({ name, age }) {
  // Varje UserCard får sin HELT EGNA räknare i minnet
  const [clicks, setClicks] = useState(0);

  return (
    <div className="user-card" style={{ border: '1px solid #ccc', margin:
'10px', padding: '10px' }}>
      <h3>{name}</h3>
      <p>Ålder: {age}</p>
      <button onClick={() => setClicks(clicks + 1)}>
        Klickad {clicks} gånger
      </button>
    </div>
  );
}
```

2. Moduler: Import, Export och Komponentstrukturer

I React bygger vi applikationen av många små filer. För att filer (moduler) ska kunna prata med varandra och använda varandras komponenter, använder JavaScript två viktiga nyckelord: **export** och **import**.

Default Export

Varje fil kan ha en "default" export. När du importerar den i en annan fil kan du döpa den till vad du vill, och du behöver inte använda måsvingar.

Exempel: `import MyCard from './UserCard';`

Named Export

Du kan exportera hur många specifika saker (funktioner, variabler) du vill från en fil. När dessa importeras måste de ligga i måsvingar och heta exakt samma sak. Exempel: `import { useState, useEffect } from 'react';`

Tre sätt att skriva och exportera en komponent

JavaScript erbjuder flera sätt att skriva samma sak. Vilken du väljer handlar ofta om personlig smak eller teamets kodstandard, men alla tre gör exakt samma jobb under huven.

```
/* --- 1. Sättet: Exportera direkt (Vanligast och kortast) --- */
export default function UserCard() {
  return <div>Hej</div>;
}

/* --- 2. Sättet: Definiera först, exportera i botten --- */
// Tydligt om man vill se vad filen exporterar genom att kolla längst ner
function UserCard2() {
  return <div>Hej</div>;
}
export default UserCard2;

/* --- 3. Sättet: Pilfunktion (Arrow Function) --- */
// Moderna pilfunktioner kräver att man definierar variabeln (const) först,
// vilket gör att vi MÅSTE exportera den på en egen rad efteråt.
const UserCard3 = () => {
  return <div>Hej</div>;
};
export default UserCard3;
```

3. Styling i React

När du skriver JSX kan du styla dina komponenter på flera sätt, men det finns ett par viktiga skillnader jämfört med vanlig HTML.

className istället för **class**

Eftersom ordet `class` redan betyder något specifikt i JavaScript (för att bygga objekt-klasser), använder JSX ordet `className` för CSS-klasser. Det fungerar sedan exakt som vanligt mot dina CSS-filer.

Inline Styling

Ibland vill man styla direkt på elementet. I vanlig HTML skriver man en textsträng: `style="color: red; margin-top: 10px;"`. I React skickar man in ett **JavaScript-objekt**. Egenskaper med bindestreck skrivs om till camelCase (t.ex. `marginTop` istället för margin-top).

```
import './Button.css'; // Här importerar vi en extern CSS-fil

export default function StyledButton() {
  return (
    <div>
      /* 1. Styling med CSS-klasser (className) */
      <button className="btn btn-primary">Normal Knapp</button>

      /* 2. Inline styling (Observera de dubbla måsvingarna!)
       Första { } betyder "nu kommer JavaScript".
       Andra { } är själva JavaScript-objektet. */
      <button style={{
        backgroundColor: 'blue',
        borderRadius: '8px',
        color: 'white'
      }}>
        Inline Knapp
      </button>
    </div>
  );
}
```

4. Dataflöde: Hur data skickas in och ut

I React finns det en strikt regel: **Data flyter alltid nedåt**. En förälder kan skicka data till sina barn, men ett barn kan inte skicka data uppåt på samma direkta sätt. Vi löser detta med *Props* och *Callbacks*.

Att skicka data NEDÅT (Props)

```
// Barnet tar emot 'title' som en Prop från föräldern  
function Header({ title }: { title: string }) {  
  return <h1>{title}</h1>;  
}
```

Att skicka data UPPÅT (Callbacks)

Föräldern skickar ner en **funktion** som en Prop. När barnet är redo att skicka upp data (t.ex. när man fyllt i ett sökfält), anropar den funktionen och skickar med datan som ett argument.

```
// 1. BARNKOMPONENTEN
interface SearchProps {
  onSearch: (searchTerm: string) => void;
}

function SearchBar({ onSearch }: SearchProps) {
  const [inputValue, setInputValue] = useState("");

  const handleSave = () => {
    onSearch(inputValue); // Vi anropar förälderns funktion med vårt värde!
  };

  return (
    <div>
      <input
        onChange={(e) => setInputValue(e.target.value)}
        value={inputValue}
      />
      <button onClick={handleSave}>Sök</button>
    </div>
  );
}

// 2. FÖRÄLDRARKOMPONENTEN
export default function App() {
  const handleChildData = (term: string) => {
    console.log("Barnet sökte på:", term);
  };

  return (
    <div>
      <SearchBar onSearch={handleChildData} />
    </div>
  );
}
```

5. Rendering av Listor och Arrayer (map)

JSX använder inte traditionella for-loopar. Istället använder vi JavaScripts inbyggda `.map()` för att omvandla en array med data till en array med React-komponenter.

Den viktiga "key"-propen

React **kräver** att varje element i en loop får en helt unik `key`. React använder denna för att förstå exakt vilken rad som eventuellt tas bort eller uppdateras senare, vilket gör om-renderingar extremt snabba.

```
const cars = [
  { id: 101, brand: "Volvo", model: "V60" },
  { id: 102, brand: "Tesla", model: "Model 3" }
];

export default function CarList() {
  return (
    <ul>
      {cars.map((car) => (
        {/* 'key' MÅSTE finnas på det yttersta elementet i loopen! */}
        <li key={car.id}>
          {car.brand} - {car.model}
        </li>
      ))}
    </ul>
  );
}
```

6. React i kombination med TypeScript

Moderna React-applikationer skrivs nästan uteslutande i **TypeScript (TS)**. TypeScript är ett lager ovanpå JavaScript som tillför ett statiskt typsystem.

- **Självdokumenterande kod:** Vi kan definiera vilka *props* en komponent förväntar sig.
- **IntelliSense:** Kodredigeraren ger perfekt autokomplettering.
- **Färre buggar:** TS varnar om vi skickar fel typ av data.

```
interface UserCardProps {
  name: string;
  age: number;
  isAdmin?: boolean; // Frivillig prop
}

export function UserCard({ name, age, isAdmin }: UserCardProps) {
  return (
    <div className="card">
      <h3>{name}</h3>
      <p>Ålder: {age}</p>
    </div>
  );
}
```

7. Djupdykning: useState (Komponentens minne)

För att en React-komponent ska kunna komma ihåg information mellan renderingar använder vi `useState`. Den returnerar alltid:

1. **Den nuvarande state-variabeln** (värdet du vill spara).
2. **En uppdateringsfunktion** (funktionen du använder för att ändra värdet).

```
const [count, setCount] = useState<number>(0);

const handleIncrease = () => {
  // "Funktionell uppdatering" garanterar att vi alltid utgår
  // från det absolut senaste värdet i minnet.
  setCount(prevValue => prevValue + 1);
};
```


8. JavaScript-mönster i React: Spread & Rest (...)

Spread-operatören ("Sprida ut")

Används för att packa upp värden ur en befintlig array eller ett objekt till ett nytt. Detta är **kritiskt** i React eftersom vi *aldrig får mutera (ändra) state direkt*.

```
const [user, setUser] = useState({ name: "Anna", age: 30 });

const handleBirthday = () => {
  // Vi "sprider" in alla gamla värden från 'user', men skriver över 'age'
  setUser({ ...user, age: user.age + 1 });
};
```

Rest-operatören ("Samla ihop")

När `...` används som funktionsparameter samlar den ihop alla *resterande* värden till en ny variabel.

```
// 'title' och 'children' plockas ut. Allt annat hamnar i 'rest'
export function CustomCard({ title, children, ...rest }: CardProps) {
  return (
    <div className="card-container" {...rest}>
      <h2>{title}</h2>
      <div>{children}</div>
    </div>
  );
}
```

9. Sidoeffekter (useEffect)

useEffect()

Hanterar saker som händer utanför den normala render-cykeln, t.ex. timers eller API-anrop. Den tar emot en **array av beroenden (dependency array)** som avgör när effekten ska köras om.

```
const [seconds, setSeconds] = useState(0);

useEffect(() => {
  const timerId = setInterval(() => {
    setSeconds(prev => prev + 1);
  }, 1000);

  // CLEANUP: Körs automatiskt när komponenten tas bort från skärmen
  return () => clearInterval(timerId);
}, []); // <- Tom array betyder: "Kör detta EXAKT en gång vid start"
```

10. Fallgropar & Anti-mönster

Direkt Mutation av State

Du får **aldrig** ändra en state-variabel direkt (t.ex. `user.name = "Olle"`). React tittar på minnesreferensen. Om du muterar direkt upptäcker inte React ändringen. *Lösningen är alltid Spread-operatorn.*

State-uppdateringar under Rendering

Att råka anropa en set-funktion direkt när JSX-koden tolkas. Detta leder till en oändlig loop.

```
// FEL! Detta anropar funktionen DIREKT under renderingen.  
<button onClick={setCount(count + 1)}>Klicka</button>  
  
// RÄTT! Vi skickar in en funktion (callback) som triggas vid klick.  
<button onClick={() => setCount(count + 1)}>Klicka</button>
```